

Universidad Sergio Arboleda
Facultad de Ingeniería

Informe de Laboratorio
Taller Práctico 2: Filtros en Video con CUDA
Guillermo Andres De Mendoza Corrales

Antony Camilo Barahona Peña

Marzo 2026

Resumen

En este informe se documenta el desarrollo e implementación de un sistema de procesamiento de video mediante filtros digitales acelerados en GPU usando CUDA a través de la librería Numba en Python. El sistema permite al usuario seleccionar uno o varios de los cinco filtros disponibles (inversión de colores, desenfoque, recorte, binarización y detección de bordes Sobel), y los aplica de forma segmentada sobre un video de entrada, generando un video de salida procesado. La arquitectura sigue tres pasos principales: extracción de frames, procesamiento con kernels CUDA y reconstrucción del video.

Índice

1. Introducción	3
2. Entorno de Desarrollo	3
3. Arquitectura del Sistema	3
4. Paso 1: Extracción de Frames	4
4.1. Descripción	4
4.2. Implementación	4
4.3. Análisis	5
5. Paso 2: Selección de Filtros	5
5.1. Descripción	5
5.2. Lógica de Segmentación	5
5.3. Restricción del Filtro de Recorte	5
6. Paso 3: Kernels CUDA	6
6.1. Filtro 1: Inversión de Colores	6
6.2. Filtro 2: Desenfoque (Blur)	6
6.3. Filtro 3: Recorte	6
6.4. Filtro 4: Binarización	7
6.5. Filtro 5: Detección de Bordes (Sobel)	7
7. Paso 4: Procesamiento por Segmento	7
8. Paso 5: Reconstrucción del Video	8
9. Resultados	8
10. Conclusiones	8

1. Introducción

El procesamiento de video es una extensión natural del procesamiento de imágenes: un video no es más que una secuencia de imágenes (frames) reproducidas a una velocidad determinada (FPS). Esto lo convierte en un problema altamente paralelizable, donde cada frame puede procesarse de forma independiente y cada píxel dentro de un frame también puede tratarse en paralelo.

CUDA (Compute Unified Device Architecture) es la plataforma de computación paralela de NVIDIA que permite ejecutar código directamente en la GPU. En este taller se utilizó Numba, una librería de Python que permite decorar funciones con `@cuda.jit` para compilarlas y ejecutarlas en GPU sin necesidad de escribir código C/C++.

El objetivo del taller fue construir un pipeline completo de procesamiento de video en tres etapas: partir el video en frames individuales, aplicar el filtro deseado a cada frame mediante kernels CUDA, y reconstruir el video procesado. Se implementaron cinco filtros distintos, seleccionables por el usuario, que se aplican de forma segmentada: el video se divide en tantos segmentos iguales como filtros se seleccionen, y cada segmento recibe un filtro diferente.

2. Entorno de Desarrollo

El desarrollo se realizó en Google Colaboratory con aceleración de hardware GPU tipo Tesla T4 (CUDA 13.0, Driver 580.82.07). El entorno utilizado fue:

- Python 3.12
- Numba (con soporte CUDA)
- NumPy
- OpenCV (cv2)

Para habilitar la compatibilidad entre versiones de Numba y CUDA se incluyó la siguiente configuración:

```
1 from numba import config
2 config.CUDA_ENABLE_PYNVJITLINK = 1
```

3. Arquitectura del Sistema

El sistema se estructura en cinco celdas de notebook, cada una con una responsabilidad clara:

1. **Celda 0 - Configuración:** instalación de dependencias y verificación de GPU con `nvidia-smi`.

2. **Celda 1 - Extracción de frames:** carga del video y partición en imágenes individuales guardadas en disco.
3. **Celda 2 - Selección de filtros:** interfaz de usuario para elegir filtros y configurar sus parámetros.
4. **Celda 3 - Kernels CUDA:** definición de todos los kernels de procesamiento en GPU.
5. **Celda 4 - Procesamiento por segmento:** aplicación del filtro correspondiente a cada frame según su segmento.
6. **Celda 5 - Reconstrucción:** unificación de los frames procesados en un video de salida descargable.

Esta separación permite reutilizar los kernels definidos en la Celda 3 para cualquier combinación de filtros sin duplicar código.

4. Paso 1: Extracción de Frames

4.1. Descripción

El primer paso consiste en descomponer el video en sus frames individuales y guardarlos en disco. Esta estrategia es recomendada para evitar agotar la RAM de la instancia de Colab al procesar videos largos, ya que los frames se leen y escriben de uno en uno durante el procesamiento.

4.2. Implementación

OpenCV se encarga de decodificar el video fotograma a fotograma. Se registran los metadatos del video (FPS, ancho y alto) para usarlos en la reconstrucción final.

```
1 cap = cv2.VideoCapture(video_name)
2 fps = cap.get(cv2.CAP_PROP_FPS)
3 width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
4 height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
5
6 frame_count = 0
7 while True:
8     ret, frame = cap.read()
9     if not ret:
10         break
11     frame_path = os.path.join(frames_dir, f"frame_{frame_count:05d}.jpg")
12     cv2.imwrite(frame_path, frame)
13     frame_count += 1
14
15 cap.release()
```

4.3. Análisis

La decodificación del video (H.264, MP4) es un proceso secuencial que no puede paralelizarse con CUDA, ya que cada frame comprimido depende de los anteriores. Por esta razón, esta etapa se realiza con OpenCV en CPU. El procesamiento paralelo con GPU se reserva para la etapa de aplicación de filtros, donde sí existe independencia total entre frames y entre píxeles.

5. Paso 2: Selección de Filtros

5.1. Descripción

Antes de procesar, el usuario selecciona qué filtros desea aplicar ingresando sus números separados por coma. El sistema solicita los parámetros adicionales únicamente para los filtros que los requieren.

5.2. Lógica de Segmentación

El video se divide en segmentos iguales según la cantidad de filtros seleccionados. Si se seleccionan n filtros y el video tiene F frames en total, cada segmento tendrá aproximadamente F/n frames:

$$frames_por_segmento = \left\lfloor \frac{F}{n} \right\rfloor \quad (1)$$

El filtro aplicado a cada frame se determina por su índice:

$$segmento = \min \left(\left\lfloor \frac{idx}{frames_por_segmento} \right\rfloor, n - 1 \right) \quad (2)$$

El uso de $\min(\cdot, n - 1)$ garantiza que los últimos frames (que podrían quedar fuera por el redondeo) sean asignados al último segmento.

5.3. Restricción del Filtro de Recorte

El filtro de recorte modifica las dimensiones de la imagen de salida, lo que es incompatible con el `VideoWriter` cuando se mezcla con otros filtros que mantienen el tamaño original. Por esta razón, si el usuario selecciona el filtro 3 junto con otros, el sistema lo elimina automáticamente y notifica al usuario:

```
1 if 3 in filtros_seleccionados and len(filtros_seleccionados) > 1:
2     print("El filtro 3 no puede combinarse con otros. Sera ignorado")
3     filtros_seleccionados.remove(3)
```

6. Paso 3: Kernels CUDA

Todos los kernels siguen la misma estructura base: cada hilo identifica su píxel mediante `cuda.grid(2)`, verifica que esté dentro de los límites de la imagen, y realiza la operación correspondiente. La configuración del grid es uniforme en todos los filtros:

```
1 threadsperblock = (16, 16)
2 blockspergrid_x = (cols + threadsperblock[0] - 1) //
  threadsperblock[0]
3 blockspergrid_y = (rows + threadsperblock[1] - 1) //
  threadsperblock[1]
```

Las imágenes se trabajan normalizadas en el rango $[0,0, 1,0]$.

6.1. Filtro 1: Inversión de Colores

Operación puntual. El nuevo valor de cada canal es el complemento respecto a 1.0:

$$C_{negativo} = 1,0 - C_{original} \quad (3)$$

```
1 @cuda.jit
2 def invert_kernel_2D(img_color, img_inverted):
3     j, i = cuda.grid(2)
4     rows = img_inverted.shape[0]
5     cols = img_inverted.shape[1]
6     if i < rows and j < cols:
7         img_inverted[i, j, 0] = 1.0 - img_color[i, j, 0]
8         img_inverted[i, j, 1] = 1.0 - img_color[i, j, 1]
9         img_inverted[i, j, 2] = 1.0 - img_color[i, j, 2]
```

6.2. Filtro 2: Desenfoque (Blur)

Operación de vecindad. Cada píxel toma el promedio de sus vecinos en una ventana de $N \times N$:

$$P_{out}(i, j) = \frac{1}{count} \sum_{di=-r}^r \sum_{dj=-r}^r P_{in}(i + di, j + dj) \quad (4)$$

El divisor `count` se ajusta dinámicamente para los píxeles del borde que tienen menos vecinos válidos.

6.3. Filtro 3: Recorte

Cada hilo en la imagen recortada lee su píxel correspondiente en la imagen original aplicando un offset:

$$(i_{original}, j_{original}) = (i + y_start, j + x_start) \quad (5)$$

Los parámetros se validan en CPU antes de lanzar el kernel para evitar accesos fuera de memoria.

6.4. Filtro 4: Binarización

Se calcula la intensidad promedio y se compara con el umbral normalizado $T_{norm} = T/255$:

$$P_{out} = \begin{cases} 1,0 & \text{si } \frac{R+G+B}{3} > T_{norm} \\ 0,0 & \text{en otro caso} \end{cases} \quad (6)$$

6.5. Filtro 5: Detección de Bordos (Sobel)

Requiere dos kernels en secuencia. Primero se convierte a escala de grises:

$$I_{gray} = 0,2989 \cdot R + 0,5870 \cdot G + 0,1140 \cdot B \quad (7)$$

Luego se aplican los kernels Sobel 3×3 para calcular el gradiente:

$$G = \sqrt{G_x^2 + G_y^2} \quad (8)$$

La raíz cuadrada se calcula con `cuda.libdevice.sqrtf()`, el método optimizado de CUDA para punto flotante. Entre ambos kernels se utiliza `cuda.synchronize()` para garantizar que el primer kernel haya completado su escritura antes de que el segundo comience a leer.

7. Paso 4: Procesamiento por Segmento

En cada iteración del loop de frames se determina qué filtro aplicar según el segmento al que pertenece el frame. El frame se carga desde disco, se normaliza, se transfiere a la GPU, se procesa y se guarda de vuelta en disco:

```

1 for idx, fname in enumerate(frame_files):
2     segmento = min(idx // frames_por_segmento, total_filtros - 1)
3     filtro_actual = filtros_seleccionados[segmento]
4
5     frame = cv2.imread(os.path.join(frames_dir, fname))
6     img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB).astype(np.float32)
7     / 255.0
8
9     # ... aplicar kernel segun filtro_actual ...

```

```
10     frame_out = (img * 255.0).astype(np.uint8)
11     frame_out_bgr = cv2.cvtColor(frame_out, cv2.COLOR_RGB2BGR)
12     cv2.imwrite(os.path.join(frames_output_dir, fname),
        frame_out_bgr)
```

8. Paso 5: Reconstrucción del Video

Los frames procesados se leen en orden y se ensamblan con `cv2.VideoWriter` usando los mismos metadatos del video original (FPS, resolución). El video resultante se descarga directamente desde Colab con `files.download()`.

```
1 fourcc = cv2.VideoWriter_fourcc(*'mp4v')
2 out = cv2.VideoWriter(output_video_name, fourcc, fps, (out_width,
    out_height))
3
4 for fname in frame_files:
5     frame = cv2.imread(os.path.join(frames_output_dir, fname))
6     out.write(frame)
7
8 out.release()
```

9. Resultados

El sistema fue probado con el video *Super Mario Galaxy: La Película - Tráiler Final* en resolución 720p (1280x720), con las siguientes características:

- **Frames extraídos:** 3437
- **FPS:** 24.0
- **Resolución:** 1280x720

Se realizaron dos pruebas principales:

Prueba 1 - Todos los filtros encadenados: se seleccionaron los filtros 1, 2, 4 y 5 de forma encadenada (cada filtro aplicado sobre el resultado del anterior). El video resultante mostró únicamente los bordes en blanco sobre fondo negro, ya que el filtro Sobel, al ser el último, eliminó toda la información de color producida por los filtros anteriores.

Prueba 2 - Filtros por segmento: se seleccionaron los filtros 1, 2, 4 y 5 en modo segmentado. El video se dividió en 4 segmentos de aproximadamente 859 frames cada uno, mostrando en cada sección del video un efecto visual claramente diferenciado: negativo, desenfoque, blanco y negro, y detección de bordes respectivamente.

10. Conclusiones

- La estrategia de guardar frames en disco en lugar de mantenerlos en RAM permite procesar videos de mayor duración sin alcanzar el límite de memoria

de Colab.

- La decodificación de video es inherentemente secuencial y no puede paralelizarse con CUDA; el beneficio de la GPU se concentra en la etapa de procesamiento píxel a píxel.
- La aplicación de filtros en modo segmentado, en lugar de encadenado, produce resultados más informativos visualmente, ya que cada sección del video muestra el efecto de un único filtro.
- El filtro de recorte requiere tratamiento especial al combinarse con otros filtros, ya que altera las dimensiones de salida y rompe la compatibilidad con el `VideoWriter`.
- `cuda.synchronize()` es fundamental cuando se encadenan múltiples kernels que tienen dependencia de datos, como en el caso del filtro Sobel.